Java Notes: Bitwise Operators

Java's *bitwise* operators operate on individual bits of integer (int and long) values. If an operand is shorter than an int, it is promoted to int before doing the operations.

It helps to know how integers are represented in binary. For example the decimal number 3 is represented as 11 in binary and the decimal number 5 is represented as 101 in binary. Negative integers are store in *two's complement* form. For example, -4 is 1111 1111 1111 1111 1111 1111 1111 1100.

The bitwise operators

Operator	Name	Example	Result	Description
`a & b`	and	3 & 5	1	1 if both bits are 1.
`a | b`	or	3 | 5	7	1 if either bit is 1.
`a ^ b`	xor	3 ^ 5	6	1 if both bits are different.
`~a`	not	~3	-4	Inverts the bits.
`n << p`	left shift	3 <<< 2	12	Shifts the bits of *n* left *p* positions. Zero bits are shifted into the low-order positions.
`n >> p`	right shift	5 >> 2	1	Shifts the bits of *n* right *p* positions. If *n* is a 2's complement signed number, the sign bit is shifted into the high-order positions.
`n >>> p`	right shift	-4 >>> 28	15	Shifts the bits of *n* right *p* positions. Zeros are shifted into the high-order positions.

Use: Packing and Unpacking

A common use of the bitwise operators (shifts with *ands* to extract values and *ors* to add values) is to work with multiple values that have been encoded in one int. Bit-fields are another way to do this. For example, let's say you have the following integer variables: age (range 0-127), gender (range 0-1), height (range 0-128). These can be packed and unpacked into/from one short (two-byte integer) like this (or many similar variations).

```
int   age, gender, height;
short packed_info;
. . .
// packing
packed_info = (((age << 1) | gender) << 7) | height;
. . .
// unpacking
height = packed_info & 0x7f;
gender = (packed_info >>> 7) & 1;
age    = (packed_info >>> 8);
```

Use: Setting flag bits

Some library functions take an int that contains bits, each of which represents a true/false (boolean) value. This saves a lot of space and can be fast to process. [needs example]

Use: Shift left multiplies by 2; shift right divides by 2

On some older computers it was faster to use shift instead of multiply or divide.

```
y = x << 3;         // Assigns 8*x to y.
y = (x << 2) + x;   // Assigns 5*x to y.
```

Use: Flipping between on and off with xor

Sometimes *xor* is used to flip between 1 and 0.

```
x = x ^ 1;       // Or the more cryptic x ^= 1;
```

In a loop that will change x alternately between 0 and 1.

Obscure use: Exchanging values with xor

Here's some weird code. It uses *xor* to exchange two values (x and y). This is translated to Java from an assembly code program, where there was no available storage for a temporary. Never use it; this is just a curiosity from the museum of bizarre code.

```
x = x ^ y;
y = x ^ y;
x = x ^ y;
```

Don't confuse && and &

Don't confuse `&&`, which is the short-circuit *logical and*, with `&`, which is the uncommon *bitwise and*. Although the bitwise *and* can also be used with boolean operands, this is extremely rare and is almost always a programming error.